



Africa Shield AI

Last-Mile Alert AI — Flood Early Warning Prototype

Built for the **AI for All Hackathon: Building Inclusive Solutions
for Early Warning and Disaster Resilience**

Organized by the African Youth Advisory Board on Disaster Risk Reduction (AYAB-DRR)
under the African Union

Project Overview & Documentation · Beginner-Friendly Edition
Prepared 10 August 2026

TOC Table of Contents

1. What is Africa Shield AI?	3
2. The problem we're solving	4
3. What we built for the hackathon	5
4. How it works — the workflow, step by step	6
5. The risk score, explained simply	7
6. Multi-language alerts	8
7. Simulated alert delivery	9
8. Technology stack, explained	10
9. System architecture	11
10. API overview	12
11. Project status & roadmap	13
12. Team & roles	14

HOW TO READ THIS DOCUMENT

Every technical term is explained in plain language the first time it appears. Boxes like this one call out things worth knowing before you continue reading. No prior software or AI background is assumed.

1 What is Africa Shield AI?

Africa Shield AI is a platform designed to predict natural disasters — floods, droughts, heatwaves, wildfires, cyclones, and earthquakes — across Africa, and to warn at-risk communities *before* the disaster happens. The design priority is **accessibility**: it is built to reach people who don't have a smartphone or reliable internet, not just people who do.

That's the big, long-term vision. For this hackathon, the team is not attempting to build all of that at once. Instead, we built one small, complete, working slice of it — end to end — so it can actually be demonstrated live, rather than just described in slides.

WHY START SMALL ON PURPOSE

A working demo of one real feature is more convincing — to judges and to future users — than a slide deck describing ten unbuilt features. Everything else the big vision promises is on an honest, documented roadmap (see Section 11), not hidden or pretended-away.

The one slice we built: "Last-Mile Alert AI" for flooding

We picked **flooding** as the first hazard because it's one of the most frequent and damaging disasters across African cities, and because the two inputs that predict flood risk — rainfall and river level — are simple enough to reason about, explain to a judge in one sentence, and demo live.



Input

Rainfall in the last 24 hours, and current river water level, for a given location.



Processing

A transparent formula turns those two numbers into a risk level: LOW, MEDIUM, or HIGH.



Translation

The warning message is generated in English and in one of three local languages.



Delivery (simulated)

A screen shows exactly what the SMS, USSD, or WhatsApp message would look like.

All four of those pieces are real and working today — not mockups of an idea, but actual running code, described in the rest of this document.

2 The problem we're solving

Early warning systems save lives — but only if the warning actually reaches the person who needs to act on it, in a language they understand, on a device they actually have. Many existing flood-warning tools assume a smartphone, a data plan, and an app download. For a large share of the population in flood-prone African regions, none of those three assumptions hold.

Plain-language term — "Last mile" In logistics and telecoms, the "last mile" is the final, hardest stretch of a journey — getting a package to an actual doorstep, or a signal to an actual phone. We use it the same way here: not "can we compute a risk score," but "does the warning actually arrive to the person standing in the flood's path."

Three barriers this project is designed around

BARRIER	WHAT IT MEANS	HOW WE ADDRESS IT
No smartphone	A warning that only shows up in an app is invisible to someone without one.	Alerts are designed to work as plain SMS or USSD text — no app required.
No reliable internet	A dashboard-only warning system misses anyone offline when the flood hits.	SMS/USSD need only a basic cell signal, not a data connection.
Language	An English-only warning is not equally useful to everyone at risk.	Every alert is generated in English <i>and</i> a local language.

WHAT "SIMULATED" MEANS, AND WHY THAT'S FINE FOR NOW

This prototype doesn't send a real text message through a paid SMS provider — it shows a screen depicting *exactly* what that message would say and look like. That's a deliberate, honest scoping decision for this hackathon (deadline 29 August 2026), not a shortcut we're hiding. Wiring up a real provider (e.g. Africa's Talking, Twilio) is a documented next step.

3 What we built for the hackathon

The scoped build is a complete pipeline for one hazard (flooding), covering nine sample African cities, with four working pieces:

PIECE	STATUS	WHAT IT DOES
Risk model	WORKING	A simple, explainable formula turns rainfall + river level into LOW / MEDIUM / HIGH.
Backend API	WORKING	Accepts a location's readings and returns the risk level plus a ready-to-send alert.
Translation	WORKING	Generates the same alert in English plus Swahili, Arabic, or Somali by country.
Simulated delivery	WORKING	Shows the exact text a person would receive by SMS, USSD, or WhatsApp.
Web dashboard	IN PROGRESS	Visual map/list of monitored cities, color-coded by risk, being built now.

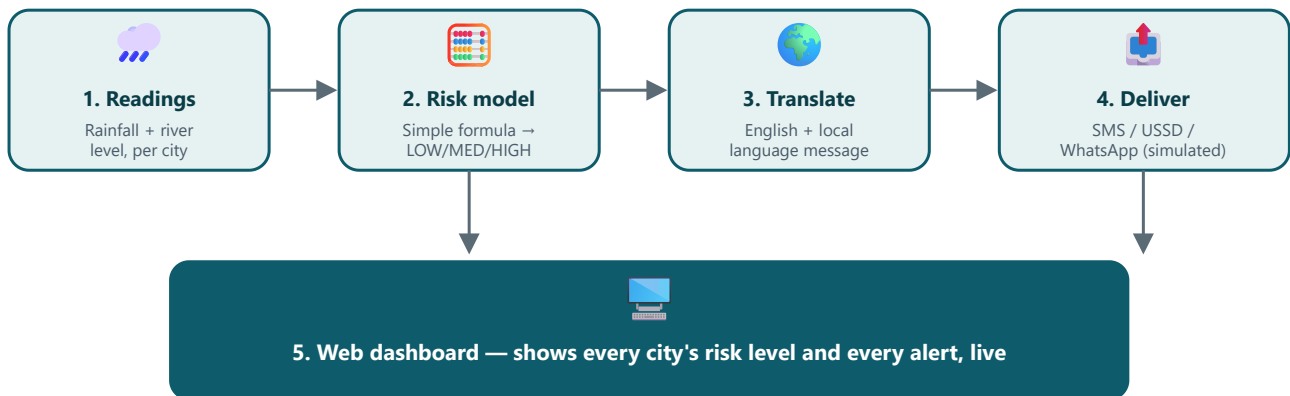
The nine monitored sample cities

To make the demo feel real rather than hypothetical, nine real African cities carry realistic sample rainfall/river readings, deliberately chosen to produce a varied, believable mix of risk levels rather than one uniform result:

CITY	RAINFALL (24H)	RIVER LEVEL	RESULT
Lagos, Nigeria	85 mm	3.2 m	HIGH
Kampala, Uganda	90 mm	3.5 m	HIGH
Dar es Salaam, Tanzania	70 mm	2.8 m	HIGH
Cairo, Egypt	35 mm	2.0 m	MEDIUM
Accra, Ghana	60 mm	2.5 m	MEDIUM
Mogadishu, Somalia	55 mm	2.6 m	MEDIUM
Nairobi, Kenya	12 mm	1.1 m	LOW
Maputo, Mozambique	15 mm	1.0 m	LOW
Kinshasa, DRC	25 mm	1.5 m	LOW

4 How it works — the workflow, step by step

Here is the entire pipeline, from a rainfall reading to a warning a person could receive, drawn as one continuous flow:



Steps 1–4 run inside the backend (FastAPI). Step 5 is the frontend (React), reading the same data.
Every step above is real, running code today — none of this is a mockup of a future idea.

BEGINNER NOTE — WHAT'S AN "API"?

Think of an API as a waiter in a restaurant. The dashboard (the customer) doesn't walk into the kitchen (the risk model) itself — it asks the waiter (the API) for something, the waiter fetches the answer, and brings it back in a standard format. That's what `POST /api/risk-check` in this project does: "here's the rainfall and river level for Lagos — what's the risk?"

5 The risk score, explained simply

The team deliberately chose a **rules-based** formula instead of a "black-box" machine-learning model for this stage. The whole point of a rules-based model is that anyone — a judge, a disaster-response official, a curious teenager — can check the math by hand and see exactly why a location got the risk level it did.

Plain-language term — "Rules-based" vs. "Machine Learning" A *rules-based* model is a formula a human wrote and can explain in one sentence (e.g. "average the two numbers, then check which zone they land in"). A *machine-learning* model instead learns its own pattern from historical data, which is often more accurate at scale — but much harder for anyone to audit or explain on the spot. We use rules today for transparency and speed; a trained ML model is a planned future upgrade (Section 11).

The formula, in one sentence

Take rainfall as a percentage of a 100 mm/24h "maximum," take river level as a percentage of a 4 m "maximum," average the two, and check which zone the result falls in.

COMBINED SCORE	RISK LEVEL
0.70 and above	HIGH
0.40 to 0.69	MEDIUM
Below 0.40	LOW

Worked example — Lagos, Nigeria

Rainfall: 85 mm of 100 mm cap → 85%



River level: 3.2 m of 4 m cap → 80%



Combined score: $(85\% + 80\%) \div 2 = 0.82$



WHY THIS MATTERS FOR TRUST, NOT JUST FOR JUDGES

Because the formula is simple and public, a local disaster-response authority could look at a HIGH rating and independently verify it — "yes, 85mm and 3.2m checks out as high risk" — rather than being asked to simply trust an opaque score. That auditability is a feature, not a limitation.

6 Multi-language alerts

A warning that only exists in a language someone doesn't read is not much of a warning. Every alert this system generates comes in **English**, always, plus one additional language chosen by the country the alert is for.

Country → language mapping

Kenya · Tanzania · Uganda Rains fall in a region where Swahili is a widely understood common language. Swahili	Egypt Arabic is the national language. Arabic	Somalia Somali is the national language. Somali
Nigeria · Ghana · Mozambique · DRC · anywhere else not listed above No single dominant local language cleanly fits Swahili, Arabic, or Somali — rather than guess, these fall back to English only. English		

Side-by-side example — Cairo, Egypt (MEDIUM risk)

ENGLISH	Flood risk is MEDIUM in Cairo. Stay alert and monitor local updates.
ARABIC (LOCAL)	خطر الفيضانات متوسط في Cairo. تَوَخَّ الحذر وتابع التحديثات المحلية.

HONEST CAVEAT — WORTH KNOWING

The Swahili, Arabic, and Somali wording shown throughout this prototype was AI-drafted for the hackathon and has not yet been checked by a native speaker. The team decided to ship these as working translations for the demo rather than delay on a review pass — a reasonable trade-off given the hackathon timeline, but one worth being upfront about if asked.

Why Arabic looks different on screen Arabic is read right-to-left, unlike English, Swahili, or Somali. The dashboard needs to render Arabic text in that direction — it's a small but easy-to-miss detail when building a multi-language interface.

7 Simulated alert delivery

This is the piece that makes the "last-mile" promise concrete rather than theoretical: a screen showing *exactly* what a person without a smartphone would actually receive.

Three channels, three different feels

CHANNEL	WHAT IT LOOKS LIKE ON A REAL PHONE	WHO IT REACHES
SMS	A plain text message bubble — works on any phone made in the last 20 years.	Broadest possible reach; no internet needed, no app needed.
USSD	A plain numbered-menu text screen (like checking mobile airtime balance) — no rich formatting at all.	Works on the most basic feature phones, including ones with no SMS inbox app.
WhatsApp	A chat bubble with timestamp, like a normal WhatsApp message.	Anyone with WhatsApp already installed — very common across Africa even on modest smartphones.

WHY "SIMULATED" IS AN HONEST WORD HERE, NOT A EUPHEMISM

No real telecom SMS gateway is connected this week — sending real texts costs money per message and requires a paid account with a provider like Africa's Talking or Twilio. The simulated screen shows the same exact message text and channel type a real integration would send — the only difference is money doesn't leave anyone's account yet. Wiring up a real provider is a documented, contained next step, not a fundamental redesign.

Example — what actually gets shown

LOCATION	RISK	MESSAGE	CHANNEL
Lagos, Nigeria	HIGH	"Flood risk is HIGH in Lagos. Move to higher ground and avoid riverbanks."	SMS (simulated)
Kampala, Uganda	HIGH	"Flood risk is HIGH in Kampala. Move to higher ground and avoid riverbanks."	USSD (simulated)
Accra, Ghana	MEDIUM	"Flood risk is MEDIUM in Accra. Stay alert and monitor local updates."	WhatsApp (simulated)

None of the choices below require a computer science background to understand — each one is explained in terms of the everyday job it does for this project.

Python + FastAPI (backend)

Python is a widely-used, readable programming language. FastAPI is a toolkit for building the "waiter" (API) that answers questions like "what's the risk in Lagos?" Chosen for speed of setup and because it auto-generates interactive documentation.

React + Vite (frontend)

React is a toolkit for building what you actually see and click on in a web browser. Vite is the tool that runs it quickly during development. Together, they build the dashboard: the map, the color-coded cards, the alert screens.

JSON data files

Plain, human-readable text files used to store sample rainfall/river readings and example responses while a full database isn't needed yet for a prototype this size.

Rules-based formula (not ML — yet)

A transparent, hand-checkable calculation instead of a trained AI model. See Section 5 for why that's a deliberate choice for this stage, not a missing feature.

Git + GitHub

A "track changes" system for code. It keeps a full history of every edit and lets multiple teammates work on the same project without overwriting each other's work.

Simulated SMS/USSD/WhatsApp

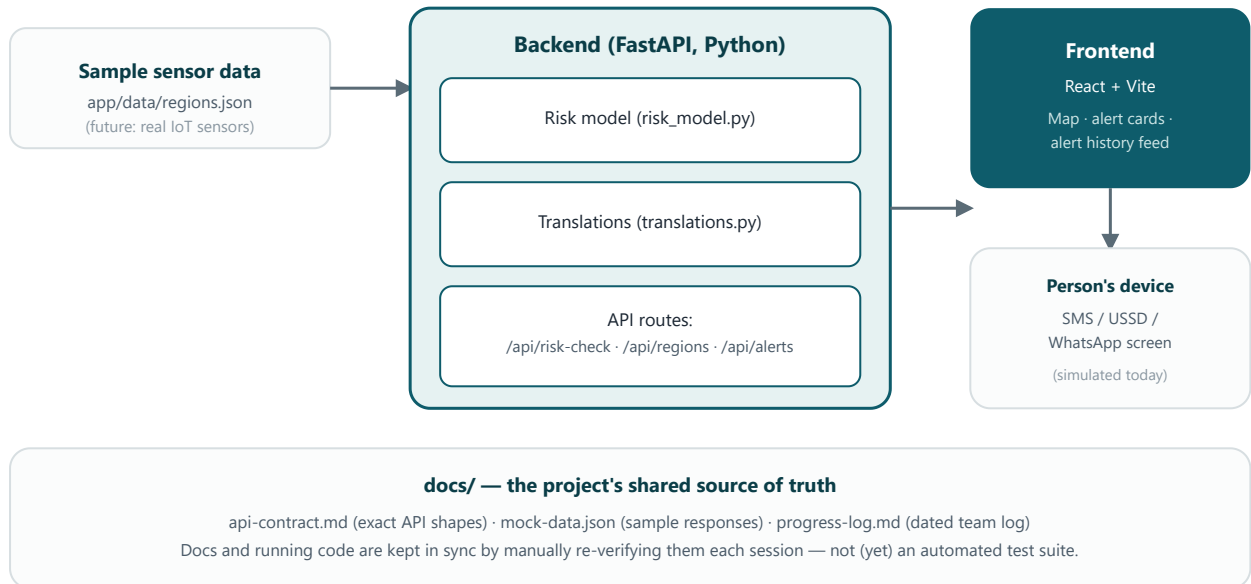
No paid third-party service is connected yet (see Section 7) — the "delivery" layer is currently a display of what would be sent, ready to be swapped for a real provider.

WHY THIS STACK, IN ONE SENTENCE

Every choice above optimizes for "fast to build correctly and easy to explain" over "maximally sophisticated" — the right trade-off for a hackathon prototype meant to prove a concept, not for a production system serving millions (that comes later, deliberately).

9 System architecture

Zooming out from the step-by-step workflow (Section 4), here's how the actual project folders and pieces relate to each other:



WHY THE DOCS/ FOLDER IS TREATED AS IMPORTANT AS THE CODE

A small team split across countries and time zones (backend vs. frontend developers) needs one agreed-upon "contract" for what data looks like, so nobody blocks on anybody else. That's what docs/api-contract.md exists for.

For readers who want a bit more technical detail: the backend exposes exactly three endpoints (think of each as a specific question you can ask the "waiter" from Section 4).

ENDPOINT	IN PLAIN ENGLISH
POST /api/risk-check	"Here's a location's rainfall and river level — tell me the risk level, score, and the alert message in English and the local language."
GET /api/regions	"List every monitored city, its coordinates, and its current risk level — for the dashboard's map/list view."
GET /api/alerts	"List the alerts that have recently been 'sent' — for the dashboard's alert history."

A real example

Sending this to POST /api/risk-check:

```
{
  "location_name": "Lagos, Nigeria",
  "rainfall_mm_24h": 85,
  "river_level_m": 3.2
}
```

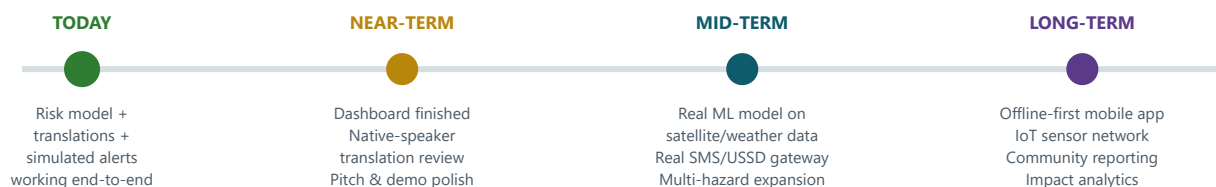
...returns exactly this:

```
{
  "risk_level": "high",
  "risk_score": 0.82,
  "alert_message_en": "Flood risk is HIGH in Lagos...",
  "local_language": "English" (Nigeria has no dedicated local-language template)
}
```

WHY THIS MATTERS FOR THE TEAM, NOT JUST OUTSIDERS

Because this exact request/response shape is written down and versioned in `docs/api-contract.md`, the person building the dashboard doesn't have to wait for the backend to be finished to start — they can build against realistic sample data from day one, then flip one setting to point at the real, running backend later.

11 Project status & roadmap



Multi-hazard = expanding beyond flooding to droughts, heatwaves, wildfires, cyclones, earthquakes
Every stage after "Today" is a documented, honest roadmap item — not a claim of what already exists.

Full future improvements list

- Replace the rules-based score with a real ML model trained on satellite (NASA/ESA) and meteorological data
- Integrate a real SMS/USSD gateway (e.g. Africa's Talking, Twilio) instead of simulating sends
- Expand beyond flooding to droughts, heatwaves, wildfires, cyclones, and earthquakes
- Build an offline-first mobile app for areas with poor connectivity
- Integrate a real translation API and expand language coverage beyond four languages
- Add community reporting ("river rising near my village") to improve accuracy
- Add low-cost IoT sensor integration (water level sensors, rain gauges)
- Add authentication for disaster-management authorities to verify alerts before they go out
- Add an impact-analytics dashboard tracking alerts sent and regions covered

12 Team & roles

NAME	ROLE
Matthias	Backend / AI — risk model, API, translation logic
Habiba	Frontend web dashboard
Farid	Frontend web dashboard
Thompson	Frontend web dashboard
Mohamed Zaki	Embedded Systems / Robotics

CLOSING NOTE

This document describes a working prototype, not a finished product — and says so honestly throughout, on purpose. Every "not yet built" item above is deliberate scoping for this hackathon's timeline, tracked openly in `docs/progress-log.md`, not something hidden from view. That transparency is itself part of the pitch: a small team that ships a real, explainable, honestly-scoped slice of a big idea, with a credible plan for the rest.